Comparación de aritméticas afines con y sin reducción k-nomial en IBEX

Informe de avance para el profesor guía

Eva Ponce Fernández · Juan Carlos Geraldo — guía: Dr. Ignacio Araya (PUCV)

30 de septiembre de 2026. Trabajo realizado entre el 13 y el 22 de septiembre. Rama raak-experimentos del repo ibex-lib, carpeta raak/.

Cómo leer esto. El informe sigue el To-Do de la reunión punto por punto (sección 2), resume todo en una página (sección 3), explica cada experimento con su gráfico (sección 4) y cierra con conclusiones y con lo que hay que revisar (secciones 5 y 6). Las rutas de archivo son relativas a `raak/` dentro del repo. Al final hay un glosario de dos párrafos por si alguna sigla no cuadra.

1. Qué había que hacer (el To-Do) 2
2. Qué se hizo en cada punto, cómo y dónde 2
2.0 Fase 0 · Verificaciones previas 2
2.1 Fase 1 · Las tres aritméticas y el k-nomial como opción independiente 3
2.2 Fase 2 · Seno y coseno 5
2.3 Fase 3 · Experimentos 6
2.4 Fase 4 · Resultados 6
3. Consolidado: qué quedó hecho 6
4. Experimentos: qué se midió, con qué, y qué salió 7
4.1 Línea base del solver sin cambios 7
4.2 Prototipo inicial del k-nomial (antes del plan) 8
4.3 El k-nomial sobre formas cuadráticas completas (QPLIB) 10
4.4 Costo de CPU del prototipo 11
4.5 Validación del código (Fase 1) 12
4.6 Seno y coseno (Fase 2) 13
4.7 Nivel 1: calidad de las cotas sin resolver (Fase 3) 15
4.8 Nivel 2: las seis variantes dentro del solver (Fase 3) 17
5. Conclusiones 21
6. Qué funciona bien y qué hay que revisar 21
7. Dónde está todo 22
Glosario mínimo 22

1. Qué había que hacer (el To-Do)

De la reunión salió esta lista:

- Implementar seno y coseno, y comparar qué tan bien aproximan la función.
- Revisar la línea 778 de "ibex affine 2" y si el plugin de Jordan Ninin está instalado.
- Separar la AA tradicional de la AF2 e implementar la revisada (RAA) completa, con su propagación de errores.
- Implementar el k-nomial sobre cada una y comparar todo: la revisada, la tradicional y la k-nomial.
- Comparar en dos niveles: solo rangos (errores de aproximación) sobre el benchmark de la librería, y dentro del solver (tiempos de resolución), con al menos media hora por instancia.
- Gráficos bien hechos. Todo ordenado, comentado, sin borrar código existente, y un `.md` que explique cada función agregada.

Esa lista se ordenó en un plan de cinco fases. Las seis variantes a comparar son las tres aritméticas, cada una con y sin k-nomial:

Fase	Qué pedía
0 · Verificaciones	Línea 778 de `ibex_Affine2_fAF2.cpp`; versión y funcionalidades del plugin de Ninin.
1 · Aritméticas	AA tradicional, AF2 y RAA activables por separado en `ibexopt-raak`; k-nomial como opción independiente sobre cada una (6 variantes); validar contención, tests del plugin y coincidencia con el Python de referencia.
2 · Seno y coseno	Implementarlos donde falten; justificar min-range vs Chebyshev vs mecanismo general; tratar intervalos con extremos y puntos de inflexión; medir sobreestimación y contención.
3 · Experimentos	Nivel 1: sobreestimación de las 6 variantes sobre el benchmark, sin resolver. Nivel 2: `ibexopt` con las 6 variantes, ≥ 30 min por instancia, estrategia de cómputo definida antes; victorias/empates/derrotas y medias geométricas separando aritmética de k-nomial.
4 · Resultados	Perfiles de desempeño, dispersiones log-log, tabla consolidada y README que explique cada gráfico.

Aritmética	Qué es	Sin k-nomial	Con k-nomial
AF2	La del plugin (`Affine2` = `AF_fAF2`): vector denso de coeficientes + un término de error.	`--arith=af2 --knomial_k=0`	`--arith=af2 --knomial_k=3`
RAA	AF2 + la parte cuadrática de la aritmética revisada del paper [A]: cuadrados de Chebyshev, forma simétrica, corner test de errores.	`--arith=raa --knomial_k=0`	`--arith=raa --knomial_k=3`
AA tradicional	`Affine3` = `AF_fAFFullI`: lista dispersa de símbolos; cada operación no afín crea un símbolo nuevo (Stolfi).	`--arith=aa --knomial_k=0`	`--arith=aa --knomial_k=3`

2. Qué se hizo en cada punto, cómo y dónde

Regla seguida en todo: nada del código original se borra. Cada cambio en C++ va marcado con el comentario `[raak local]`, entra como parche numerado en `patches/` (cabecera "Qué cambia / Por qué / Verificado") y se activa con un interruptor; apagado, el código original corre bit a bit igual. Los parches se aplican sobre dos repos fijados a un commit: `ibex-team/ibex-affine` (`c219009`, el plugin de Ninin) y el fork `rilianx/ibex-lib` (`5e51fffc`, el único con `ibexopt04`). El inventario completo de funciones está en `docs/CAMBIOS_CODIGO.md`.

2.0 Fase 0 · Verificaciones previas

Línea 778. El único archivo con "Affine2" en el nombre y más de 778 líneas es `ibex_Affine2_fAF2.cpp`. Según la versión que se mire, la línea 778 es: en el upstream sin parches, una línea de `compact(tol)`, que mueve coeficientes

pequeños al error y **no se llama desde ningún sitio**, así que no afecta; en nuestra rama con los parches 0001 y 0002, el inicio del bloque del cuadrado de Chebyshev de la RAA (`use_raak_sq`), que sí es nuestro y está validado; en el árbol actual, una línea en blanco del bucle original de `Asqr`. Detalle en `docs/FASE0_VERIFICACIONES.md`.

Plugin de Ninin. Instalado desde `ibex-team/ibex-affine`, rama `update_affine`, commit `c219009c` (4 de septiembre de 2026), versión 1.0, 11 de 11 tests OK. Trae dos modelos (`AF_fAF2` denso y `AF_fAFFullI` disperso), todas las operaciones elementales, dos modos de linealización por hilo (Chebyshev y min-range) para `exp`, `log`, `sqrt`, `pow`..., el evaluador `AffineEval` y el linearizador `LinearizerAffine2`. Lo que **no** trae y motivó la Fase 2: `sin` y `cos` tienen una sola implementación que ignora el modo.

2.1 Fase 1 · Las tres aritméticas y el k-nomial como opción independiente

Qué había que hacer. Que cada aritmética se pudiera encender sola, y el k-nomial encima de cualquiera. Antes, el prototipo mezclaba ambas cosas en un solo parámetro (`raak_k`: 0 = AF2, 1 = parte RAA, 3 = RAA + bloques).

Cómo. Dos interruptores por hilo en `affine/ibex_KNomial.h`: `affine_mode()` elige AF2 o RAA dentro de `AF_fAF2`, y `knomial_max_k()` enciende la reescritura k-nomial (0 = apagada, 3 = bloques de hasta 3 variables). La AA tradicional es el otro modelo del plugin, `AF_fAFFullI`, al que se le agregó la misma reescritura. El CLI `ibexopt-raak` fija los interruptores con `--arith` y `--knomial_k`.

Dónde y qué hace cada función.

Archivo	Función / cambio	Qué hace
`affine/ibex_KNomial.h` (nuevo, parches 0002/0004/0005)	`knomial_rewrite(Q, w, n, max_k, blocks)`	El método k-nomial: simetriza Q, arma el grafo de pares con diagonal del mismo signo y extrae con avidez bloques $c\cdot(u^Tx)^2$ de hasta `max_k` variables, descontando presupuesto diagonal. Port bit a bit de `knomial.py` (modo `sdsos`, `sign_mode='fixed'`).
	`knomial_quad_bound(x, y, n, max_k, shift, rad)`	Acota la parte cuadrática $\varepsilon^TQ\varepsilon$ de un producto de formas afines, con $Q = (xy^T+yx^T)/2$ y $\varepsilon \in [-1,1]^n$: reescribe con `knomial_rewrite` si $max_k \geq 2$ y acota bloque a bloque ($[0, (\Sigma\|u_i\|)^2]$), diagonal y cruces residuales. Con $max_k < 2$ es la forma simétrica término a término (la parte cuadrática de la RAA de [A]).
	`knomial_quad_bound_sparse(rx, ry, ...)`	Lo mismo para las listas dispersas de `AF_fAFFullI`: alinea los ids y llama a la anterior.
	`raa_corner_cross(a, b, ex, ey, em)`	Corner test de la Ec. 16 de [A]: acota los cruces entre los errores de los factores y sus partes lineales buscando el máximo en las esquinas de $[-1,1]^2$, en vez de $ex\cdot Sy + Sx\cdot ey$.
	`affine_mode()`, `knomial_max_k()`, `KNomialStats`	Los dos interruptores y los contadores de diagnóstico (cuántos productos, bloques, cotas usadas) que el CLI imprime como `RAA-K stats`.

Archivo	Función / cambio	Qué hace
ibex_Affine2_fAF2.cpp (parches 0002/0004/0005)	operator*= de AffineMain<AF_fAF2>	Antes de sobrescribir los coeficientes los copia; si ambos factores comparten ≥ 2 símbolos y hay RAA o k ≥ 2, calcula la cota alternativa y **solo la usa si es más estrecha que la de fAF2** más su margen; con RAA y errores en los factores aplica el corner test. Si nada mejora, el resultado es bit a bit el original.
	Asqr	Con RAA, la parte cuadrática $(a^T\varepsilon + e\cdot h)^2$ se acota por su rango exacto $[0, (Sx+e)^2]$ (cuadrado de Chebyshev) en vez de centro $Sx/2$ y radio $(Sx+e)^2 - Sx/2$.
ibex_Affine3_fAFFullI.cpp (parche 0006)	operator*= de AffineMain<AF_fAFFullI>	Con k ≥ 2 acota la parte no lineal con knomial_quad_bound_sparse más los cruces con los _garbage en intervalos, en vez del producto de intervalos rad(x)·rad(y); solo si estrecha.
ibex_AffineMain.h / _impl.h (0007)	nonvar_noise_sum(nb_var)	Suma de los coeficientes de los símbolos que no son variables (los rayos nuevos de la AA), que err() no incluye y la relajación necesita.
ibex_LinearizerAffine2.{h,cpp} (0001/0007)	LinearizerAffineT<T>, affine_form_is_linearizable, affine_extra_error	El linearizador del plugin parametrizado por modelo, para poder relajar también con la AA. Incluye el arreglo del segfault con formas no activas (parche 0001).
cli/cli_ibexopt_raak.cpp, cli_Optimizer04Config_Affine.{h,cpp}	main(), Optimizer04Config_Affine::get_linear_relax()	Copia adaptada de ibexopt_config04: parsea --lr=xn|art|compo, --arith, --knomial_k, --trig, fija los interruptores, elige el linearizador (X-Taylor, afín con AF2/RAA, afín con AA, o la composición compo) e imprime las estadísticas al final.
src/optim/ibex_Optimizer04Config.cpp (ibex-lib, parche 0003)	get_ctc()	Dos líneas: envuelve también las relajaciones raak/raakcompo en CtcPolytopeHull. Alias de compatibilidad.

En conjunto y por qué. El producto de dos formas afines genera una parte cuadrática en los símbolos de ruido. fAF2 la acota con una fórmula fija ($Sx\cdot Sy - Sxy/2$). La RAA la acota mejor (forma simétrica, cuadrado exacto, corner test) y el k-nomial intenta agrupar esa parte cuadrática en cuadrados de sumas. Separar las dos cosas en interruptores distintos es lo que permite medir cada efecto por separado, que era el objetivo del plan. Como cada cota nueva solo se usa si mejora, ninguna variante puede ser peor que fAF2 en un producto aislado.

Validación (punto 1.3). Programas en tests/, todos re-ejecutados el 21 de septiembre:

Test	Qué comprueba	Resultado
knomial_golden.cpp	knomial_rewrite contra la salida guardada de knomial.py (knomial_golden.txt).	80 casos, 327 bloques, idénticos.

Test	Qué comprueba	Resultado
raak_arith_test.cpp	Productos y cuadrados aleatorios de Affine2: contención, pendientes invariantes, radio nunca mayor que fAF2.	3 000 + 3 000, 0 violaciones.
raa_corner_test.cpp	Corner test: contención por muestreo y bit a bit con la referencia cuando no hay error.	10 000 pares, 0 violaciones.
aa_knomial_test.cpp	k-nomial en Affine3: contención, k = 0 bit a bit.	3 000 productos, 0 violaciones; radio k3/k0 = 0.91.
aa_linearizer_test.cpp	La relajación lineal con AA contiene a la función.	200 polinomios × 1 000 puntos, 0 violaciones.
make check del plugin	Tests upstream.	11 de 11.
Regresión del B&B (results/04-fase1-validacion/fase1-20260916/paso11)	7 instancias × variantes contra la referencia guardada antes del refactor.	35 de 35 idénticas en celdas y óptimo.

2.2 Fase 2 · Seno y coseno

Qué había que hacer. Las tres aritméticas comparten la plantilla AffineMain<T>, así que sin y cos ya existían en las tres, pero con una sola fórmula (pendiente de la cuadratura de Gauss de dos puntos) que ignora el modo Chebyshev/min-range del plugin y no coincide con la RAA de [A]. Había que implementar las dos fórmulas, justificar la elección y tratar los intervalos con extremos y puntos de inflexión.

Cómo. Se escribió un solo mecanismo general de residuo y encima las dos fórmulas; el código original queda como modo legacy para control. Parche 0008, flag --trig=auto|cheb|minrange|legacy.

Archivo	Función	Qué hace
ibex_AffineTrig.h (nuevo)	trig_residual_range(is_sin, alpha, itv, residual)	Mecanismo general (Sec. 3.2 de [A]): dada una pendiente α, el rango exacto de g(v) = f(v) − αv es el hull de g en los extremos y en todos los puntos críticos f'(v) = α dentro del intervalo. Eso hace válida la forma aunque el intervalo tenga extremos o inflexiones, sin suponer convexidad.
	trig_lin_mode(), trig_lin_resolve(), TrigLinStats	El interruptor por hilo, la traducción de auto al modo efectivo y los contadores (TRIG stats).
ibex_AffineMain_trig_impl.h (nuevo)	Asin_CH, Acos_CH	Pendiente de cuerda α = (f(b) − f(a))/(b − a) (Ec. 12 de [A], el "Chebyshev" de la RAA) + residuo general.
	Asin_MR, Acos_MR	Min-range: si hay un extremo dentro, α = 0 (la forma es el intervalo f(itv)); si no, α = f' en el extremo con menor |f'|, y la anchura de la forma es exactamente la del rango.
	Atrig_apply(alpha, residual)	Paso final común: α·forma + mid(residuo) ± rad(residuo), mismo patrón que Aexp_CH y las demás unarias.
ibex_AffineMain_impl.h	despacho en Asin / Acos	Un switch al inicio: Chebyshev → *_CH, min-range → *_MR, legacy → cae al código original, que no se toca.

Por qué así. Las fórmulas específicas dan la pendiente; el mecanismo general da el residuo exacto para cualquier pendiente. Separarlos evita duplicar la parte delicada (enumerar puntos críticos con redondeo exterior) y es lo que garantiza contención en todos los casos. Para la Fase 3 se usó auto = cuerda, porque es la definición de la RAA y es la mejor en las funciones básicas (sección 4.6). Detalle en docs/FASE2_TRIGONOMETRICAS.md.

2.3 Fase 3 · Experimentos

Nivel 1, rangos. `tests/range_eval.cpp` carga cada instancia de `benchs/optim`, evalúa objetivo y hasta 50 restricciones con las seis variantes sobre 9 cajas (la inicial y 2 sub-cajas aleatorias por cada tamaño relativo 1/2, 1/4, 1/8, 1/16, que imitan lo que ve el B&B) y compara la anchura de la forma afín con el rango muestreado (200 puntos + centro) y con la extensión natural de intervalos. Registra contención puntual y de intervalo, si la variante llegó a reemplazar alguna cota y el tiempo. Script `scripts/wsl_fase3_nivel1.sh`.

Nivel 2, solver. `scripts/wsl_fase3_sweep.sh` corre `ibexopt-raak` con las seis variantes más el control `xn` sobre cada instancia, con `-t 1800` (30 min de CPU), 12 procesos en paralelo, instancias ordenadas por dificultad, y es reanudable (una fila por corrida; una corrida interrumpida no deja fila). `wsl_fase3_status.sh` monitorea y `wsl_fase3_launch.sh` encadena las etapas. Registra estado, celdas, `[f_lb, f_ub]`, CPU y las estadísticas de cada mecanismo.

Estrategia de cómputo (definida antes de lanzar). Con la línea base de 60 s solo 2 de 133 instancias agotaban el tiempo, así que el costo lo dominan pocas instancias: ≈ 13 configuraciones × 4 instancias × 1 800 s ≈ 26 h de CPU, 2 a 3 h de pared por etapa con 12 procesos. Cuatro etapas: **A** easy+medium × `compo`, **B** easy+medium × `art`, **C** hard+others × `compo`, **D** hard+others × `art`. `blowup` y `unsolved` (63 instancias) quedaron fuera: 40 h de pared para devolver casi solo TIME_OUT.

Análisis. `analysis/analyze_range.py` (nivel 1), `analyze_sweep.py` (nivel 2: victorias/empates/derrotas con umbral ±5 %, medias geométricas sobre las instancias resueltas por ambas, perfiles de Dolan–Moré, dispersiones log-log) y `consolidate.py` (tabla consolidada). Diseño y lecturas en `docs/FASE3_EXPERIMENTOS.md`.

2.4 Fase 4 · Resultados

Cada carpeta de `results/` lleva `meta.txt` (binario, commit, parches, parámetros, hora) y un `README.md` que explica cada archivo y gráfico. `results/07-fase3-nivel2-solver/fase3-20260921/RESUMEN.md` es la tabla consolidada de las cuatro etapas y `resumen_medias_geom.png` su gráfico. `analysis/vizstyle.py` fija un color por variante (paleta validada para deficiencias de color) para que todos los gráficos se lean igual. Los gráficos de este informe se regeneraron el 27 de septiembre a partir de los mismos CSV y están en `results/08-graficos/`.

3. Consolidado: qué quedó hecho

Pieza	Qué es	Dónde
8 parches sobre `ibex-affine` + 1 sobre `ibex-lib`	0001 segfault del linearizador · 0002 prototipo k-nomial · 0003 `get_ctc` · 0004 interruptores separados · 0005 corner test · 0006 k-nomial en AA · 0007 linearizador para AA · 0008 seno/coseno.	`patches/`
`ibexopt-raak`	El optimizador con `--lr`, `--arith`, `--knomial_k`, `--trig`.	`cli/`
9 programas de prueba y evaluación	Golden, aritmética, corner, AA, linearizador, trigonometría, rangos (nivel 1), QPLIB; el de las seis variantes quedó solo en WSL.	`tests/`
Scripts WSL	Clonado, compilación, parches, tests, barridos reanudables, monitor.	`scripts/`
Análisis	5 scripts Python: trigonometría, nivel 1, nivel 2, consolidado, estilo.	`analysis/`
Resultados	Todos los CSV con su `meta.txt`, análisis y gráficos.	`results/`
Documentación	Plan y bitácora, Fase 0, Fase 2, Fase 3, cambios de código, entorno.	`docs/`
Commits	11 commits locales en la rama `raak-experimentos` (uno por fase o etapa), sin push.	`git log -- raak`

Volumen de evidencia: 124 674 evaluaciones de rangos sobre 380 instancias, 17 640 evaluaciones trigonométricas, unas 1 700 corridas del solver a 30 min (A: 132 × 7, B: 132 × 6, C: 15 × 7, D: 15 × 6), más los tests unitarios. Cero violaciones de contención en todo.

4. Experimentos: qué se midió, con qué, y qué salió

Van en orden cronológico. Los tres primeros son del prototipo inicial (antes del plan, 13 a 15 de septiembre); sirven de contexto porque motivaron la separación de la Fase 1. En todos los gráficos de celdas o CPU, **más bajo es mejor** y 1 = igual que la base.

Fuente de datos común. `benchs/optim` de IBEX: problemas de optimización global no lineal con restricciones de la biblioteca COCONUT, en formato Minibex. Conjuntos: `easy` (101), `medium` (31), `hard` (7), `others` (8), más `blowup` (25), `unsolved` (38) y `benchs-unconstrainedoptim` (180, sin restricciones y con muchas `sin`/`cos`). Son en su mayoría productos de variables sueltas con pocos términos cuadráticos densos. **Métricas del solver:** celdas = nodos que el B&B tuvo que procesar (mide la calidad de las cotas), CPU = tiempo medido por el optimizador, `[f_lb, f_ub]` = intervalo final del óptimo (verifica que todas las variantes dan la misma respuesta).

4.1 Línea base del solver sin cambios

Experimento 1 · Línea base `xn` / `art` / `compo`

Pregunta	¿Cuánto aporta la relajación afín del plugin frente a X-Taylor, antes de tocar nada?
Datos	133 instancias de `easy+medium` × 3 relajaciones, límite 60 s. `results/01-linea-base/baseline-20260913-193306/results.csv`.
Variables	Estado, celdas, CPU, `[f_lb, f_ub]`.
Cómo	Perfil de desempeño (Dolan–Moré) en celdas: para cada instancia, celdas / celdas de la mejor relajación; la curva dice qué fracción de instancias resuelve cada una dentro de un factor X de la mejor.

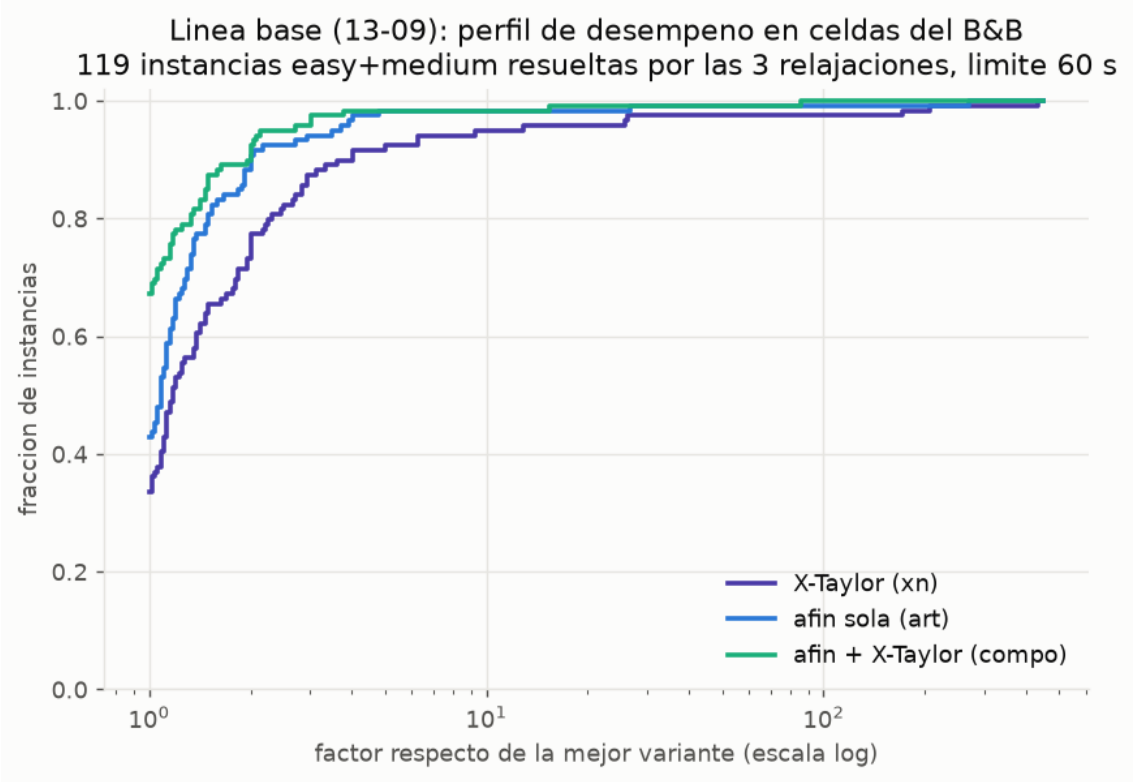


Figura 1. Perfil de desempeño en celdas de la línea base.

Qué muestra. `compo` (afín ∘ X-Taylor) está arriba en todo el rango: es la mejor o casi en el 68 % de las instancias; `art` sola en el 43 %; `xn` en el 34 % y tiene una cola larga. Óptimos compatibles en las tres.

Conclusión. La relajación afín ya aporta mucho sobre X-Taylor (media geométrica de celdas 0.69). Esto fija `compo` como relajación principal y `xn` como control para el resto del trabajo.

4.2 Prototipo inicial del k-nomial (antes del plan)

Experimento 2 · k-nomial v1, solo en productos

Pregunta	¿Reduce celdas meter la reescritura k-nomial en la multiplicación de fAF2?
Datos	`easy+medium`, 60 s, `raak/raakcompo` (k = 3) frente a `art/compo`. `results/02-prototipo-raak/baseline-20260914-120246/raak_vs_base.csv`.
Variables	Cociente de celdas con/sin k-nomial, en las 28 corridas donde la reescritura llegó a actuar.
Cómo	Un punto por corrida, ordenado; línea en 1.

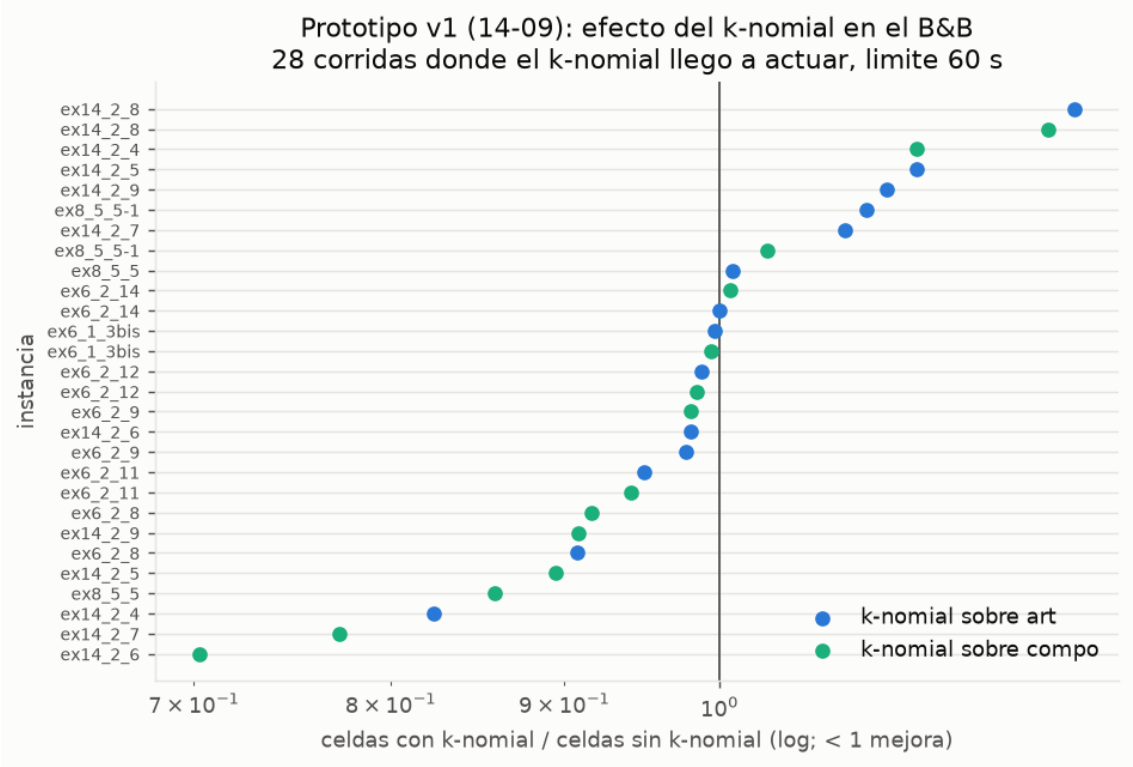


Figura 2. Prototipo v1: cociente de celdas por instancia.

Qué muestra. Con `compo` la mayoría de los puntos queda bajo 1 (7 gana, 2 pierde, media 0.95); con `art` sola se reparte (2 gana, 5 pierde, media 1.02). Solo actúa en dos familias: ex14_2_* y ex6_2_*.

Conclusión. Primer indicio de que el k-nomial solo tiene material en pocas instancias y de que su efecto depende de la relajación. También mostró que `raak` mezclaba dos cosas (parte RAA y bloques), lo que llevó al experimento 4 y a separarlas en la Fase 1.

Experimento 3 · v2: agregar los cuadrados de Chebyshev

Pregunta	¿Qué pasa al acotar también `sqr(x)` por su rango exacto (cuadrado de Chebyshev de la RAA)?
Datos	Mismo barrido con el prototipo v2. `results/02-prototipo-raak/baseline-20260914-131620/raak_v2_vs_base.csv`, 59 corridas.
Variables	Celdas de v1 y de v2, ambas divididas por la base.
Cómo	Dispersión log-log: debajo de la diagonal, v2 mejor que v1.

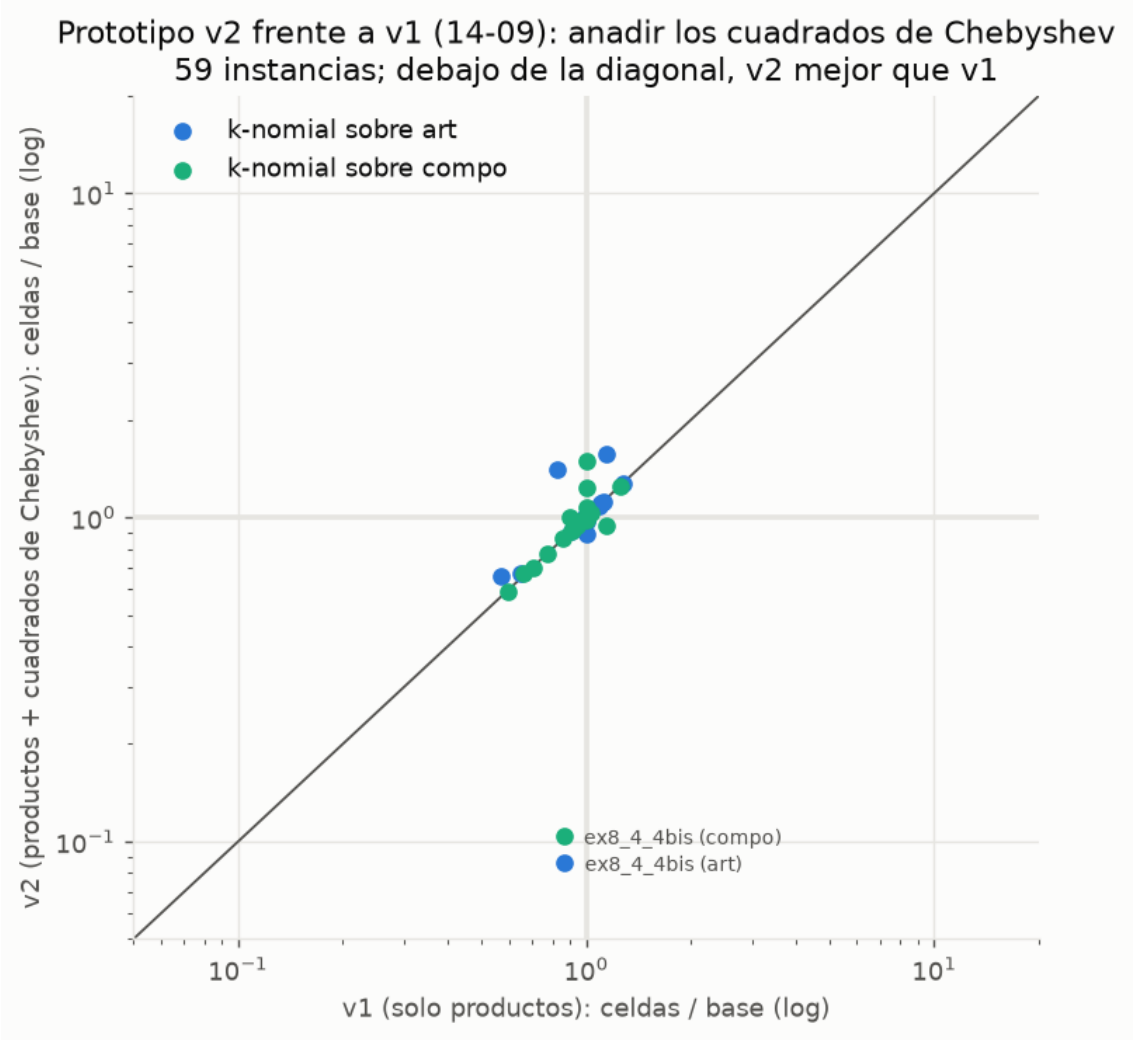


Figura 3. v2 frente a v1.

Qué muestra. Casi todos los puntos caen sobre la diagonal (los cuadrados no cambian nada) salvo `ex8_4_4bis`, que baja a 0.09 en las dos relajaciones (de 83 220 a 8 618 celdas con `compo`). La media geométrica pasa de 0.96 a 0.91.

Conclusión. Toda la ganancia de v2 viene de un solo cambio, el cuadrado exacto, y de pocas instancias. Es la primera señal de que la parte RAA pesa más que los bloques k-nomiales.

Experimento 4 · Ablación: parte RAA (k = 1) frente a RAA + bloques (k = 3)

Pregunta	Dentro del prototipo, ¿cuánto aporta la parte cuadrática de la RAA sola y cuánto los bloques?
Datos	`results/02-prototipo-raak/baseline-20260915-025226/descomposicion_k1_k3.csv`, 50 corridas comparables.
Variables	Celdas con k = 1 y con k = 3, divididas por la base sin RAA.
Cómo	Por instancia, los dos valores unidos por una línea.

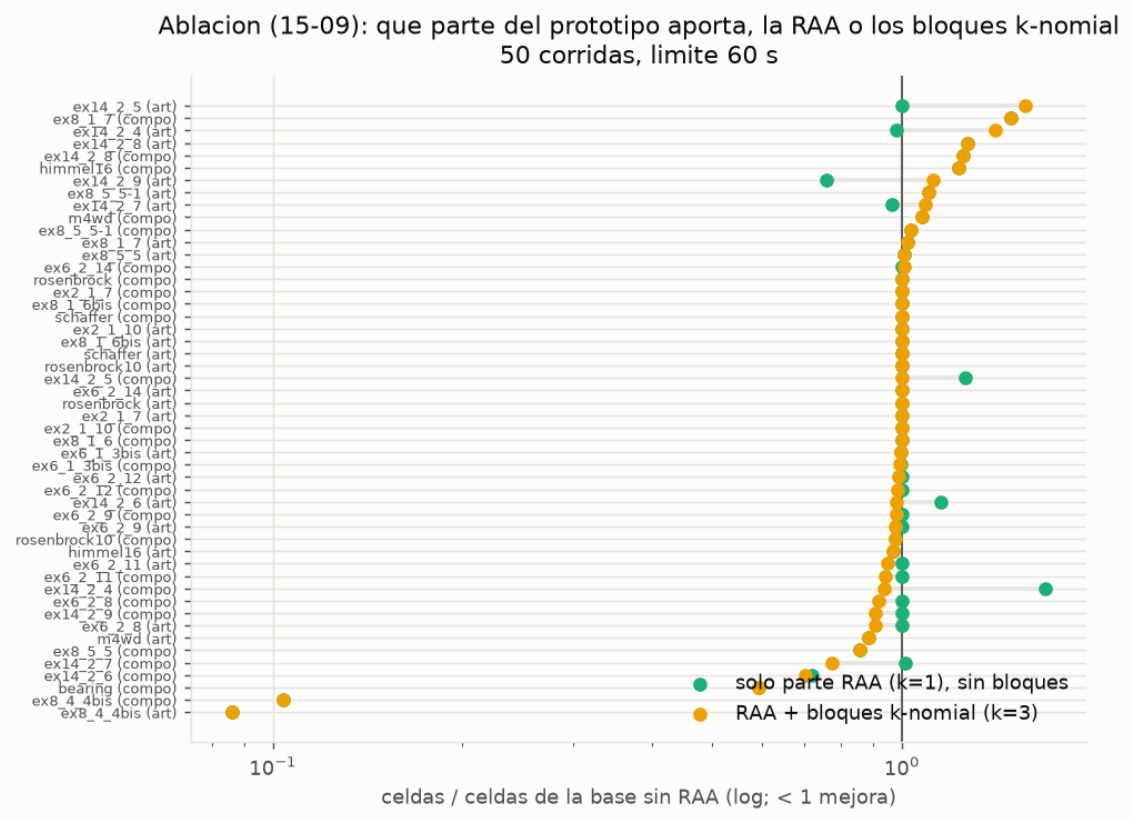


Figura 4. Ablación k = 1 frente a k = 3.

Qué muestra. Los puntos naranjos (k = 3) y verdes (k = 1) coinciden casi siempre: `ex8_4_4bis` y `bearing` mejoran igual con k = 1. Donde difieren (ex14_2_*), los bloques a veces ayudan (ex14_2_7, ex14_2_6) y a veces empeoran (ex14_2_4, ex14_2_5).

Conclusión. Confirma lo del experimento 3 y fija la pregunta correcta para la Fase 3: medir la aritmética y el k-nomial por separado.

4.3 El k-nomial sobre formas cuadráticas completas (QPLIB)

Experimento 5 · QPLIB evaluado en IBEX

Pregunta	Si se le da al método una matriz Q entera y no un producto binario, ¿cuánto estrecha la envolvente afín?
Datos	58 instancias de QPLIB (biblioteca de programas cuadráticos), cada una escrita de 4 formas por `tests/qplib_ibex_forms.py`: monomio a monomio (como en un `.bch`), reescrita con k-nomial k = 2 y k = 3 en Python, y descomposición espectral $\Sigma\ \lambda\ (v^{T}x)^{2}$. Evaluadas por `tests/qplib_affine_eval.cpp` con fAF2 (modo 0) y con el prototipo RAA-K k = 3 (modo 3). `results/03-qplib/qplib-eval-20260914-212602/eval.csv`.
Variables	Radio de la forma afín resultante; reducción = 1 − radio(modo 3) / radio(modo 0).
Cómo	Un punto por instancia y formulación; la raya negra es la media.

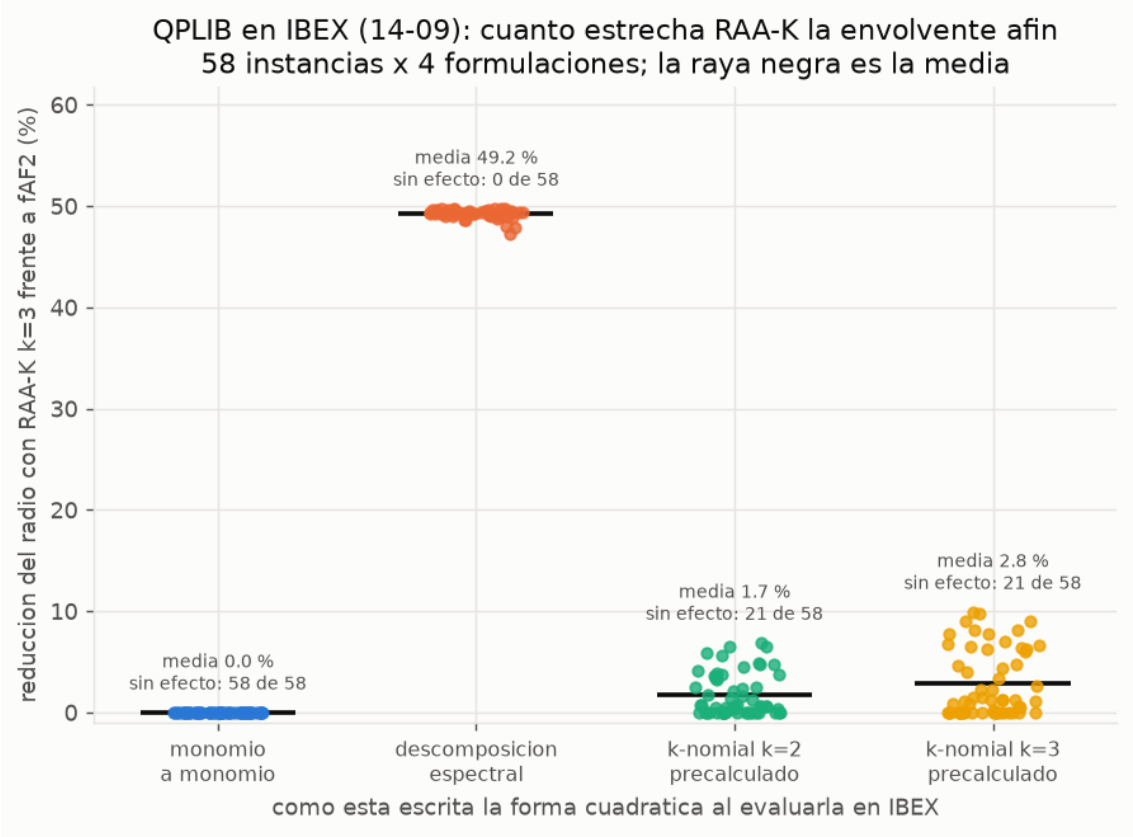


Figura 5. Reducción del radio en QPLIB según cómo esté escrita la forma cuadrática.

Qué muestra. Monomio a monomio: 0 % en las 58, porque cada producto $x_i \cdot x_j$ es un nodo de rango 2 sin símbolos compartidos y el método nunca entra. Con los bloques ya escritos (k = 2, k = 3): 1.7 % y 2.8 % de media, hasta 9.9 %, y 21 instancias sin efecto (diagonal nula). Espectral: 49 % en todas, porque la forma es una suma de cuadrados de formas lineales y el cuadrado de Chebyshev los acota exactamente.

Conclusión. El método necesita ver la forma cuadrática completa. Escrita monomio a monomio, que es como llegan los modelos a IBEX, no tiene nada que agrupar. Esto es el mismo diagnóstico que después dio el nivel 2, visto desde el otro lado. El port C++ coincide con la métrica `af2_error` del Python de referencia en las 58 instancias (diferencia relativa ≤ 1e-11), y la re-evaluación tras el refactor reprodujo exactamente los 36 valores de control (`results/04-fase1-validacion/fase1-20260916/paso15`).

4.4 Costo de CPU del prototipo

Experimento 6 · Sobrecosto de CPU

Pregunta	¿Cuánto tiempo extra cuesta la reescritura por producto?
Datos	10 instancias × {`art`, `raak:0`, `raak:3`} × 3 repeticiones, corridas secuenciales con orden rotado para evitar sesgo de caché. `results/02-prototipo-raak/timing-20260914-132113/timing.csv`.
Variables	CPU / CPU media de `art` en la misma instancia.
Cómo	Tres puntos por celda y su media.

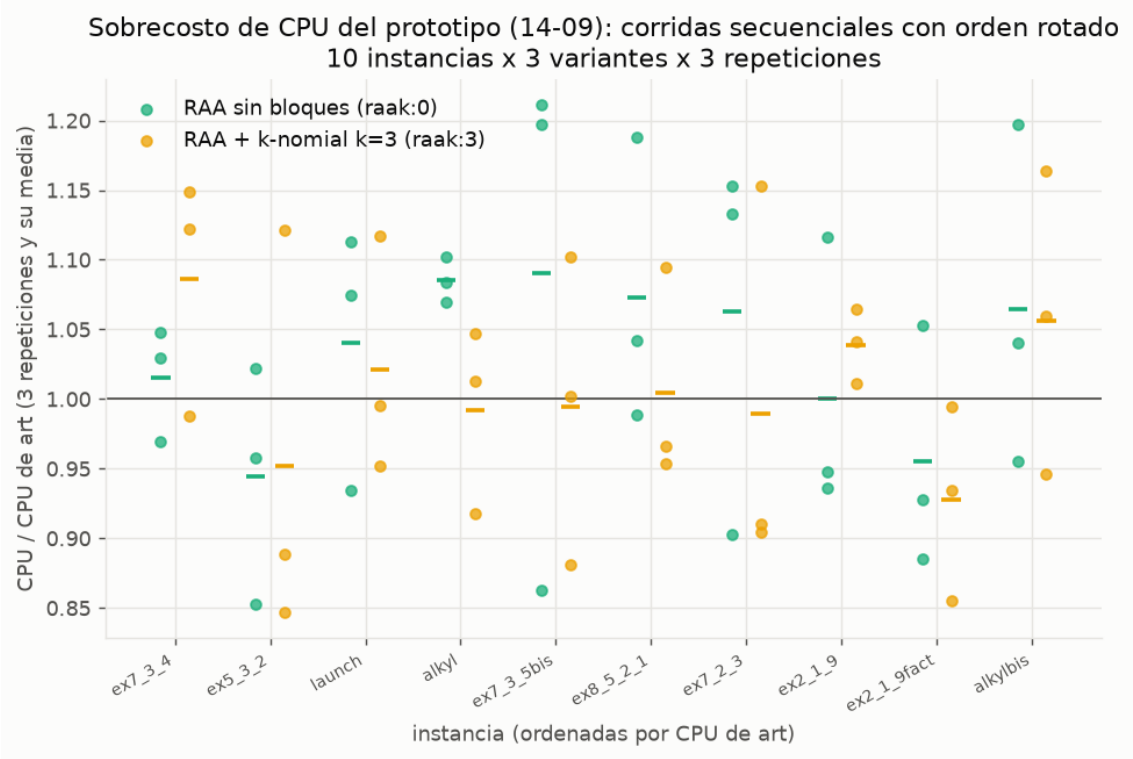


Figura 6. Sobrecosto de CPU con repeticiones.

Qué muestra. Los cocientes se mueven entre 0.85 y 1.2 sin patrón: la media geométrica es 1.03 para `raak:0` y 1.00 para `raak:3`. La dispersión entre repeticiones de la misma configuración es del mismo tamaño que la diferencia entre configuraciones.

Conclusión. El k-nomial no cuesta tiempo medible en el B&B: el LP y la propagación dominan. También fija la escala del ruido de CPU (±15 %), que es la razón del umbral de ±5 % en celdas y de usar medias geométricas en el nivel 2.

4.5 Validación del código (Fase 1)

Experimentos 7 y 8 · Regresión del refactor y pruebas de validez

Pregunta	Después de separar `--arith` y `--knomial_k`, ¿sigue todo dando lo mismo? ¿Y las cotas contienen siempre al rango real?
Datos	Regresión: 7 instancias de referencia × variantes, 300 s, contra `results/04-fase1-validacion/fase1-ref/` (celdas y óptimo guardados antes del refactor). Validez: los tests de `tests/` listados en 2.1.
Variables	Celdas por variante; violaciones de contención; diferencias bit a bit con las referencias.
Cómo	Comparación exacta, no estadística.

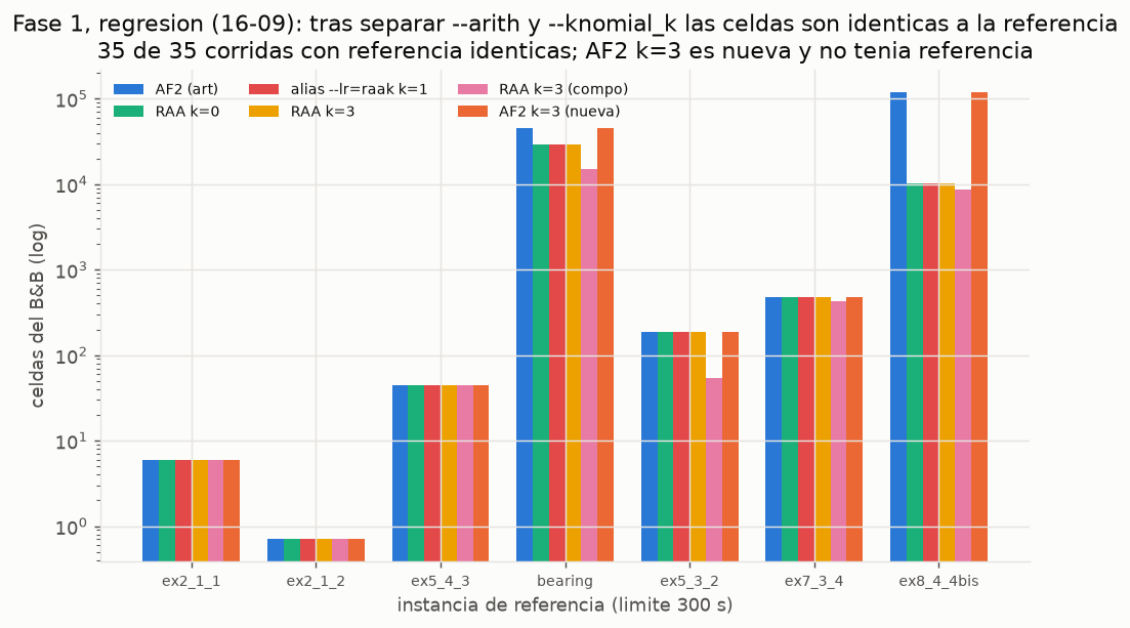


Figura 7. Regresión: celdas por instancia y variante tras el refactor.

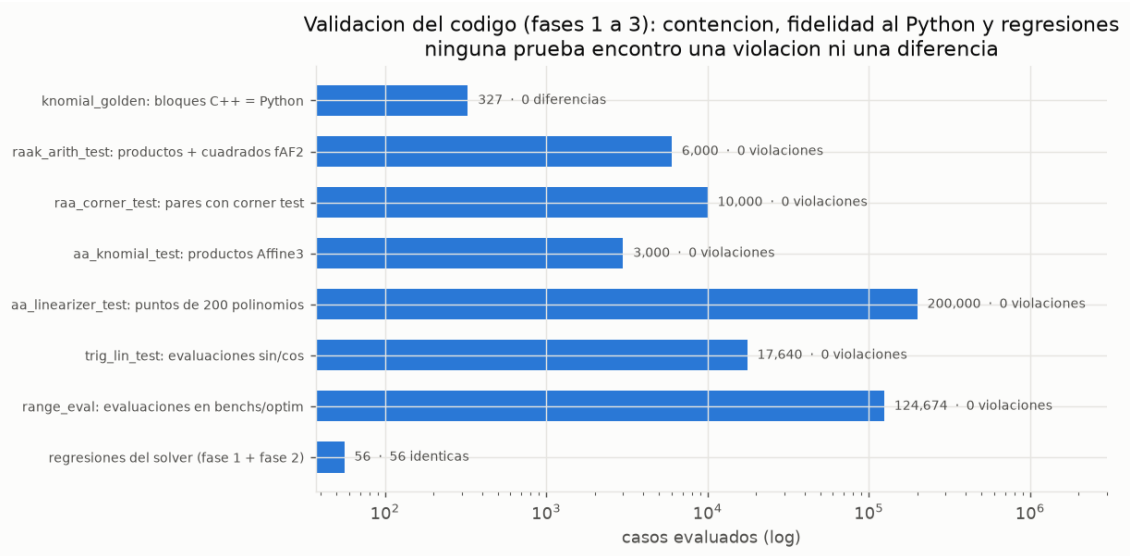


Figura 8. Todas las pruebas de validez con su tamaño y resultado.

Qué muestra. En la figura 7 cada grupo de barras es una instancia: todas las variantes con referencia dan exactamente las mismas celdas que antes (35 de 35). `bearing` y `ex8_4_4bis` son las únicas donde RAA cambia el número, y lo hace igual que antes del refactor. La figura 8 resume las pruebas: desde 327 bloques comparados con Python hasta 200 000 puntos del linearizador y 124 674 evaluaciones sobre instancias reales, sin una sola violación.

Conclusión. El código es fiel al método (bit a bit con Python) y válido (contención en todo lo probado). El refactor no cambió ningún resultado. Esto es lo que permite confiar en los experimentos de las fases 2 y 3.

4.6 Seno y coseno (Fase 2)

Experimento 9 · Calidad de las tres linealizaciones

Pregunta	¿Qué pendiente conviene: la original del plugin, la cuerda (Chebyshev de la RAA) o min-range?
Datos	`tests/trig_lin_test.cpp`: 49 centros en [−π, π] (incluyen extremos e inflexiones) × 15 anchos de 0.01 a 2π = 735 intervalos, para `sin`, `cos`, `sin·cos` y `sin²+cos²`, en AF2 y en AA, en los 3 modos; 401 puntos de muestreo por intervalo. `results/05-fase2-trigonometria/fase2-20260921/trig_lin.csv`.

Variables	Sobreestimación = anchura de la forma / anchura del rango exacto (1 = exacto); error de linealización (radio de la parte no lineal); violaciones; tiempo.
Cómo	Media geométrica por función y modo; y la misma métrica en función del ancho del intervalo.

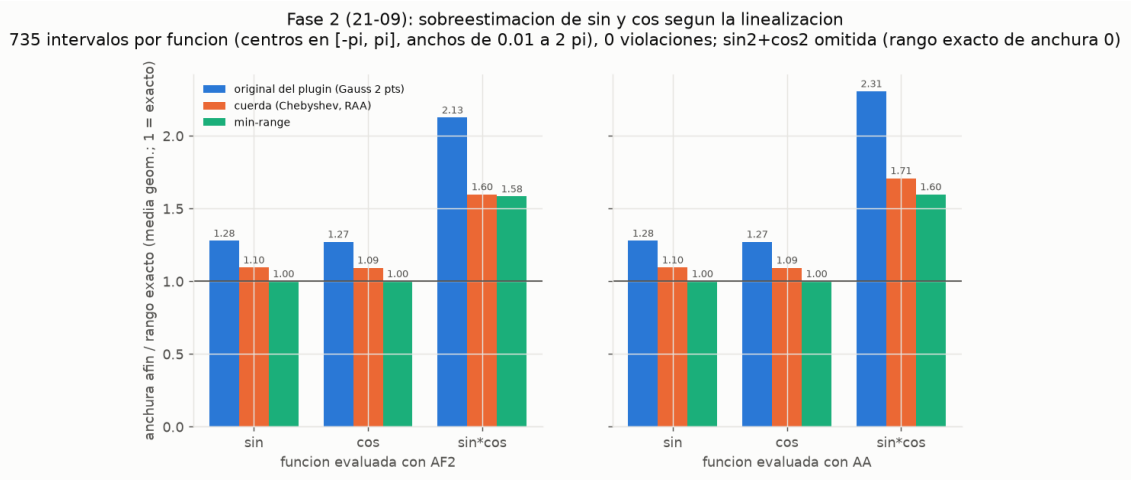


Figura 9. Sobreestimación por función y modo (AF2 y AA).

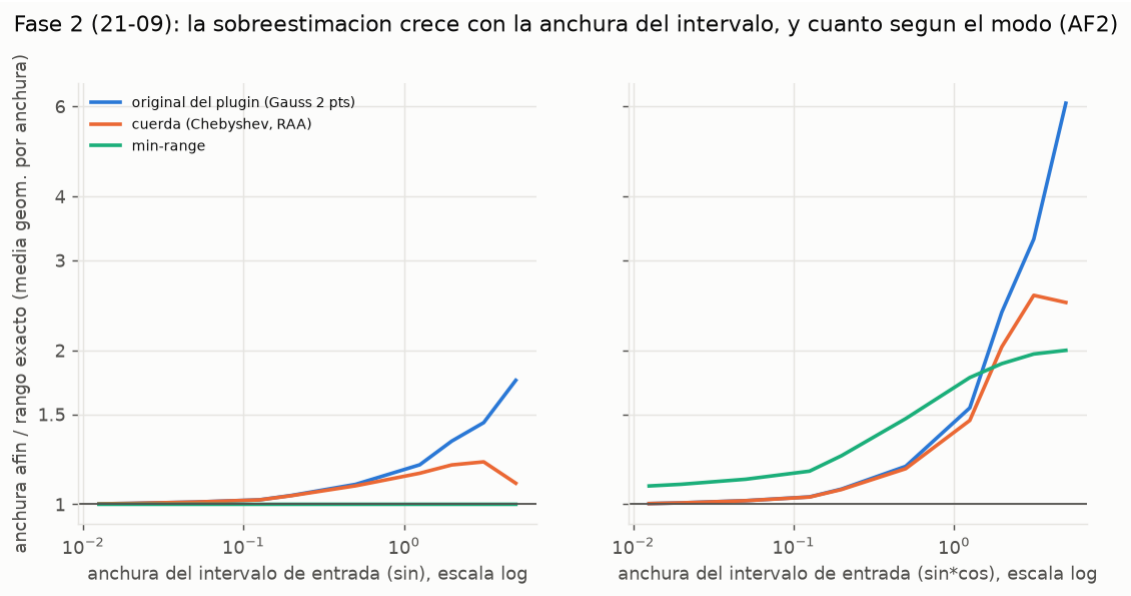


Figura 10. Sobreestimación según el ancho del intervalo de entrada.

Qué muestra. En `sin` y `cos` la cuerda baja la sobreestimación de 1.28 a 1.10 y min-range la deja en 1.00, el rango exacto. En `sin·cos` (donde ya interviene la multiplicación) las tres se separan más: 2.13, 1.60 y 1.58 en AF2; en AA la cuerda es un poco peor (1.71) porque su multiplicación pierde correlación entre rayos. La figura 10 muestra que la diferencia aparece a partir de anchos ≈ 0.3 y crece con el ancho; min-range es plano en `sin` porque degenera al intervalo apenas hay un extremo dentro. Contención: 0 violaciones en 17 640 evaluaciones. `sin²+cos²` se omite del cociente porque su rango exacto tiene anchura 0.

Conclusión. La cuerda domina al código original en todos los intervalos (error de linealización menor en 705 de 735 para `sin`, nunca mayor) y conserva correlación; min-range da el rango exacto pero su error de linealización es 35 % mayor (pierde la pendiente). Por eso la Fase 3 usa la cuerda. Resultado adicional: el mecanismo general de residuo hace válidas las tres incluso con extremos e inflexiones dentro.

Experimento 10 · Las linealizaciones dentro del solver

Pregunta	¿Se nota el modo en problemas reales con `sin/cos`?
Datos	Las instancias de `easy` con `sin/cos` (`ex14_1_4`, `ex8_1_1`, `ex8_1_2`; `ex4_1_7` no cargó), 3 modos × {`art`, `compo`} + `xn`, 120 s. `results/05-fase2-trigonometria/fase2-20260921/trig_instancias/runs.csv`.
Variables	Celdas por instancia y configuración.

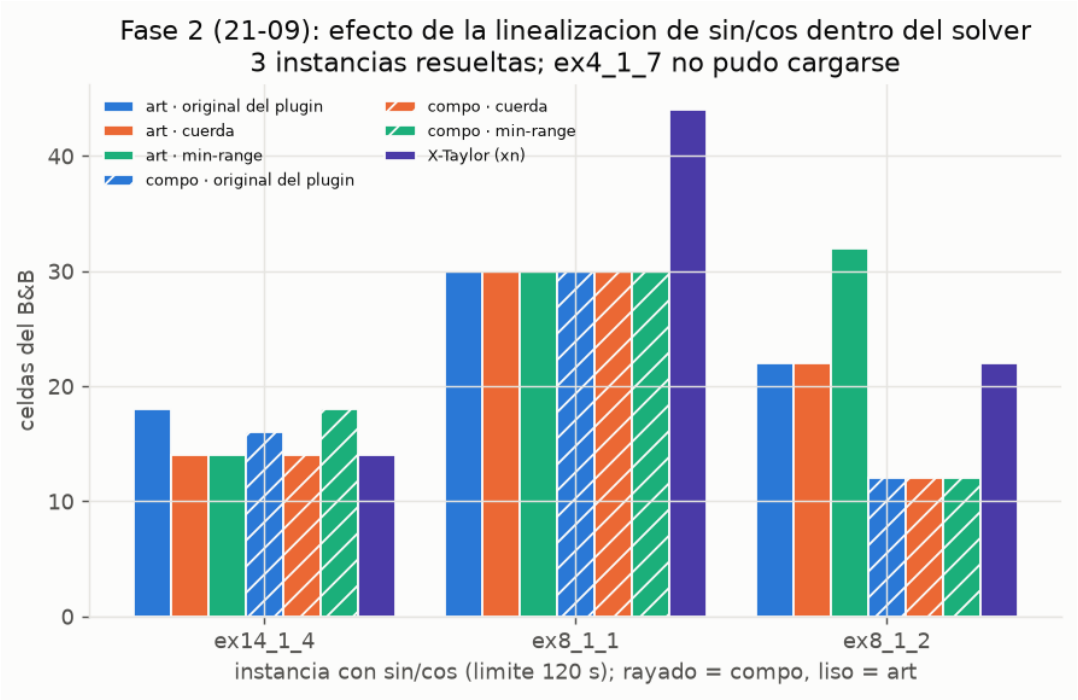


Figura 11. Celdas por instancia y modo trigonométrico.

Qué muestra. Las tres instancias son triviales (≤ 0.02 s) y solo `ex14_1_4` cambia con el modo (18 → 14 con la cuerda). La regresión sin trigonométricas, 21 corridas, fue idéntica a la referencia.

Conclusión. No hay instancias con `sin/cos` suficientemente duras en `easy+medium` para medir el efecto en el solver; la comparación relevante queda en el nivel 1 (`benchs-unconstrainedoptim`, 180 instancias con trigonometría).

4.7 Nivel 1: calidad de las cotas sin resolver (Fase 3)

Experimento 11 · Sobreestimación de las seis variantes sobre el benchmark

Pregunta	Sin B&B de por medio, ¿qué aritmética da las cotas más ajustadas y cuánto agrega el k-nomial a cada una?
Datos	380 instancias de todos los conjuntos de `benchs/optim` (10 no se pudieron parsear), objetivo y hasta 50 restricciones, 9 cajas por función: 20 608 ternas (instancia, función, caja) × 6 variantes = 124 674 evaluaciones. `results/06-fase3-nivel1-rangos/nivel1/range.csv.gz`.
Variables	Sobreestimación = anchura afín / anchura del rango muestreado; lo mismo para la extensión natural de intervalos (referencia); violaciones; en cuántas ternas actuó cada mecanismo; µs por evaluación.
Cómo	Media geométrica por variante y tamaño de caja; y conteo de ternas más estrechas o más anchas en más de 5 % por cada par de variantes (separación de efectos).

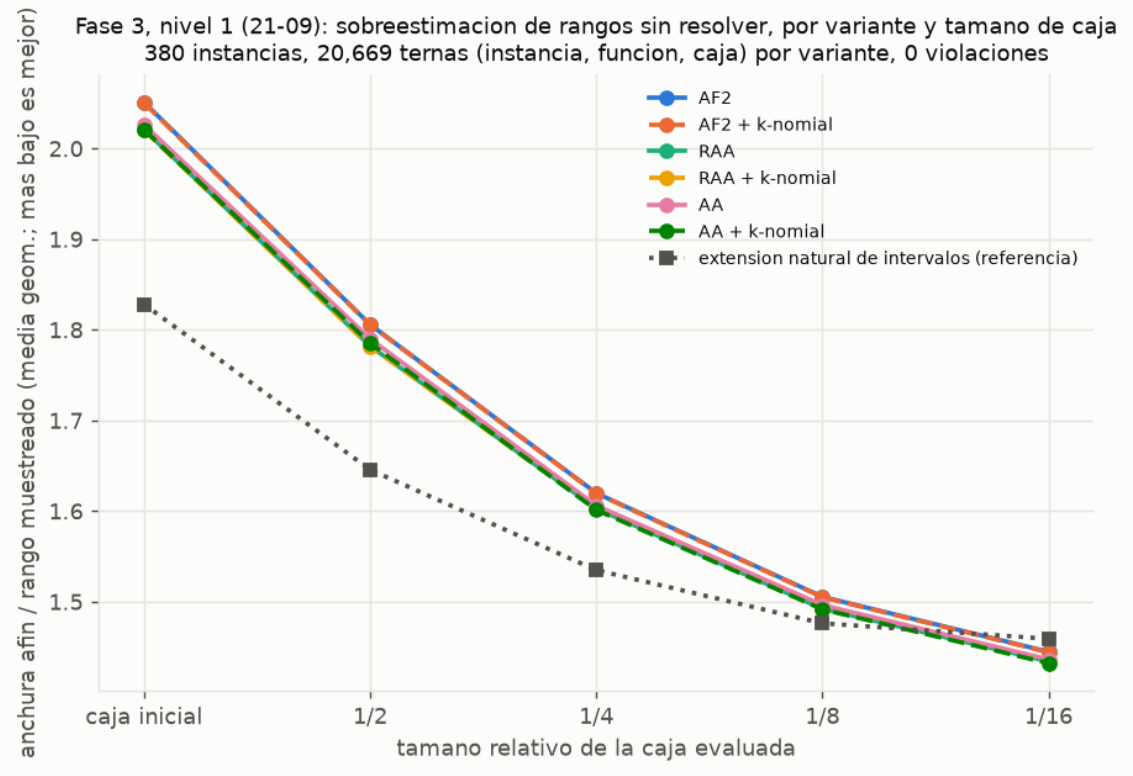


Figura 12. Sobreestimación por variante y tamaño de caja; la línea punteada es la extensión de intervalos.

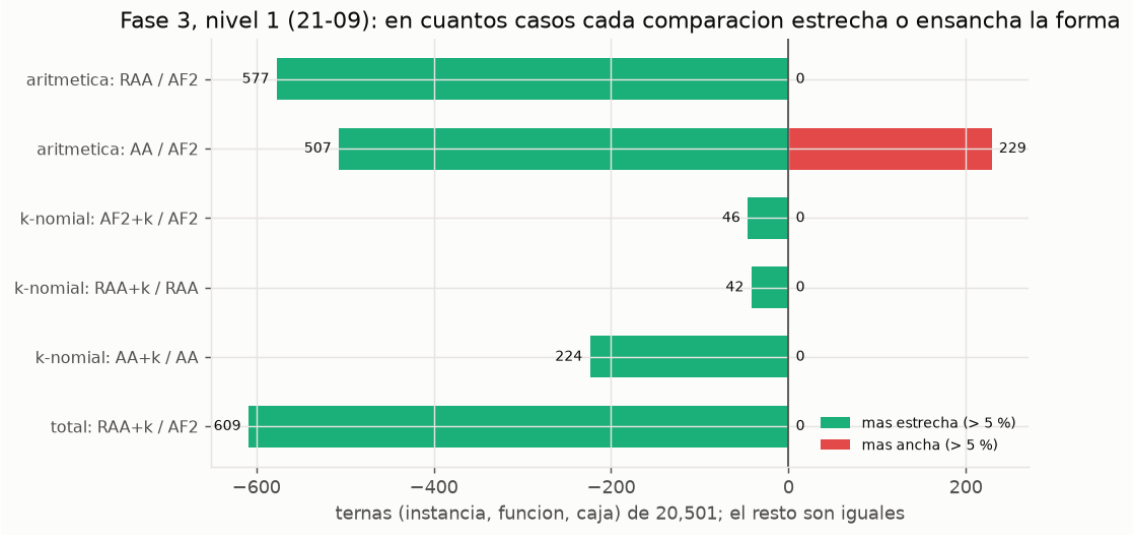


Figura 13. Ternas más estrechas o más anchas por par de variantes.

Qué muestra. Figura 12: las seis curvas van casi pegadas. RAA y AA + k-nomial son las más bajas en todas las escalas; AF2 la más alta. En la caja inicial y en 1/2 todas las afines son **peores** que los intervalos (2.05 frente a 1.83): linealizar operaciones no lineales sobre dominios anchos acumula error; desde 1/8 las afines ganan. Figura 13: RAA es más estrecha que AF2 en 577 ternas y más ancha en **0**; AA gana en 507 pero pierde en 229; el k-nomial sobre AF2 y RAA actúa en 46 y 42 ternas (de 20 501), sobre AA en 224, y nunca ensancha.

Conclusión. Tres lecturas. (1) RAA nunca es peor que AF2 y donde actúa, actúa mucho (en `benchs-unconstrainedoptim` pasa de 9.68× a 8.72×). (2) El k-nomial casi no actúa sobre AF2/RAA porque necesita dos factores que compartan ≥ 2 símbolos con diagonal del mismo signo, raro en modelos escritos monomio a monomio; sobre AA actúa 5 veces más porque los rayos nuevos dan símbolos comunes. (3) AA es 15 veces más lenta por

evaluación (1 190 µs frente a 72) y deja formas más anchas en cadenas largas. Que las afines pierdan contra los intervalos en cajas grandes justifica usar `compo` y no `art` sola.

4.8 Nivel 2: las seis variantes dentro del solver (Fase 3)

Diseño común a las cuatro etapas: `-t 1800`, 12 procesos, la base es AF2 sin k-nomial de la misma etapa; gana = cociente < 0.95, pierde = > 1.05; medias geométricas sobre las instancias resueltas por ambas. Los perfiles de desempeño muestran celdas y CPU; la figura 18 resume las cuatro etapas. Tablas completas en `results/07-fase3-nivel2-solver/fase3-20260921/RESUMEN.md`.

Experimento 12 · Etapa A: easy + medium, relajación `compo` (132 instancias × 7)

Datos	`results/07-fase3-nivel2-solver/fase3-20260921/A_easy_medium_compo/runs.csv`. Estados idénticos en las 7 configuraciones: 125 SUCCESS, 1 INFEASIBLE, 1 NO_FEASIBLE_FOUND, 4 UNBOUNDED, 1 UNREACHED_PREC (`ex6_2_6`). Óptimos compatibles.

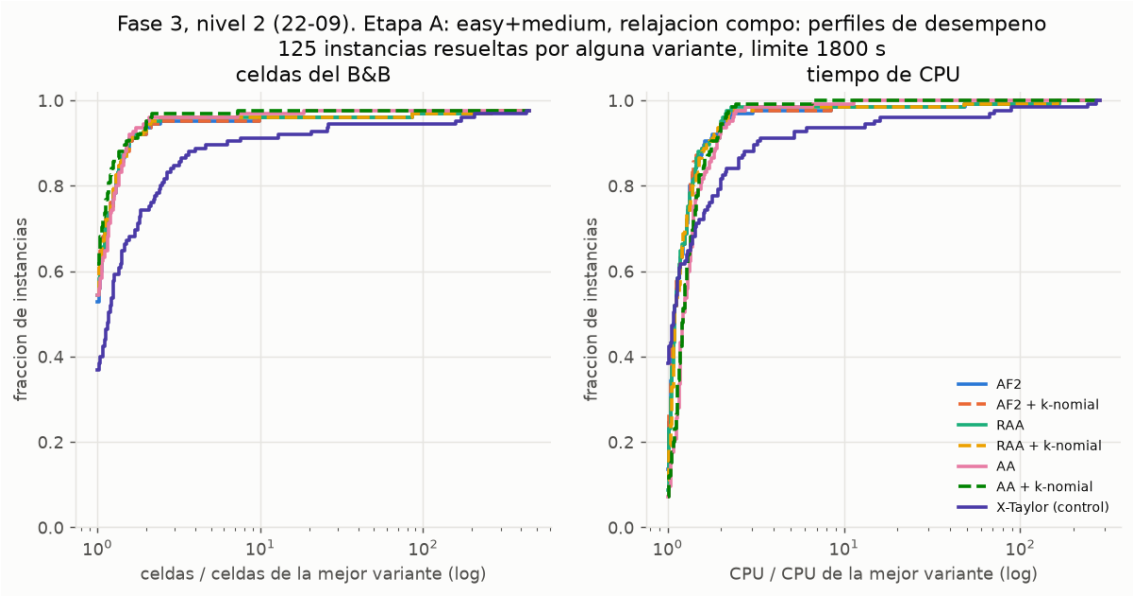


Figura 14. Etapa A: perfiles de desempeño en celdas y CPU.

Qué muestra. Las seis afines van juntas y muy por encima de xn. AA + k-nomial (verde discontinuo) es la curva más alta en celdas. Medias geométricas de celdas frente a AF2: AF2+k 0.994, RAA 0.984, RAA+k 0.982, AA 0.946, **AA+k 0.905**, xn 1.443. Dentro de cada aritmética el k-nomial da 0.994 (AF2), 0.998 (RAA) y **0.957 (AA, 24 gana / 15 pierde, CPU 0.971)**.

Conclusión. Con `compo`, la mejor variante en celdas es AA + k-nomial con CPU igual a AF2 (1.009). Sobre AF2 y RAA el k-nomial es neutro: solo entra en `ex14_2_*` (5 instancias con bloques). RAA iguala a AF2 salvo `ex8_4_4bis` (83 220 → 8 618 celdas) y `bearing` (25 704 → 15 210), las dos por los cuadrados de Chebyshev.

Experimento 13 · Etapa B: easy + medium, relajación `art` (132 × 6)

Datos	`results/07-fase3-nivel2-solver/fase3-20260921/B_easy_medium_art/runs.csv`. Mismos estados en las 6; `ex6_2_6` agota el tiempo en todas (con `art` sola es más dura: 3.8 M de celdas frente a 0.9 M con `compo`).

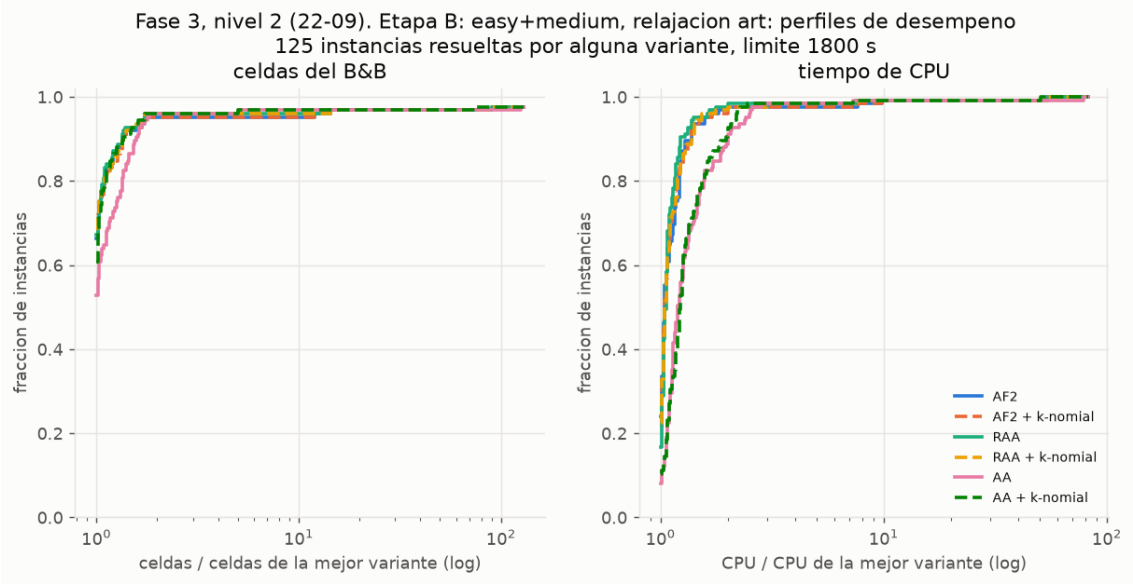


Figura 15. Etapa B: perfiles de desempeño.

Qué muestra. RAA (verde) es la curva más alta en CPU. Medias frente a AF2: RAA **0.979 celdas, 0.969 CPU** (5 gana / 4 pierde); AF2+k 1.002; RAA+k 0.984; AA 1.029 celdas y 1.159 CPU; AA+k 0.975 celdas, 1.132 CPU. k-nomial dentro de AA: 0.948 (29 gana / 7 pierde).

Conclusión. Con la relajación afín sola, RAA es la mejor y la AA tradicional empeora frente a AF2: sin la intersección con X-Taylor, la pérdida de correlación de sus rayos nuevos pesa más. El k-nomial la rescata en parte, pero sigue costando 13 % más de CPU. Sobre AF2/RAA, neutro (1.002, 1.005).

Experimento 14 · Etapa C: hard + others, relajación `compo` (15 × 7)

Datos	`results/07-fase3-nivel2-solver/fase3-20260921/C_hard_others_compo/runs.csv`. 11 instancias resolubles; `polygon25` agota el tiempo en todas, `ship` sin punto factible, 2 `test_infinity*` no acotadas.

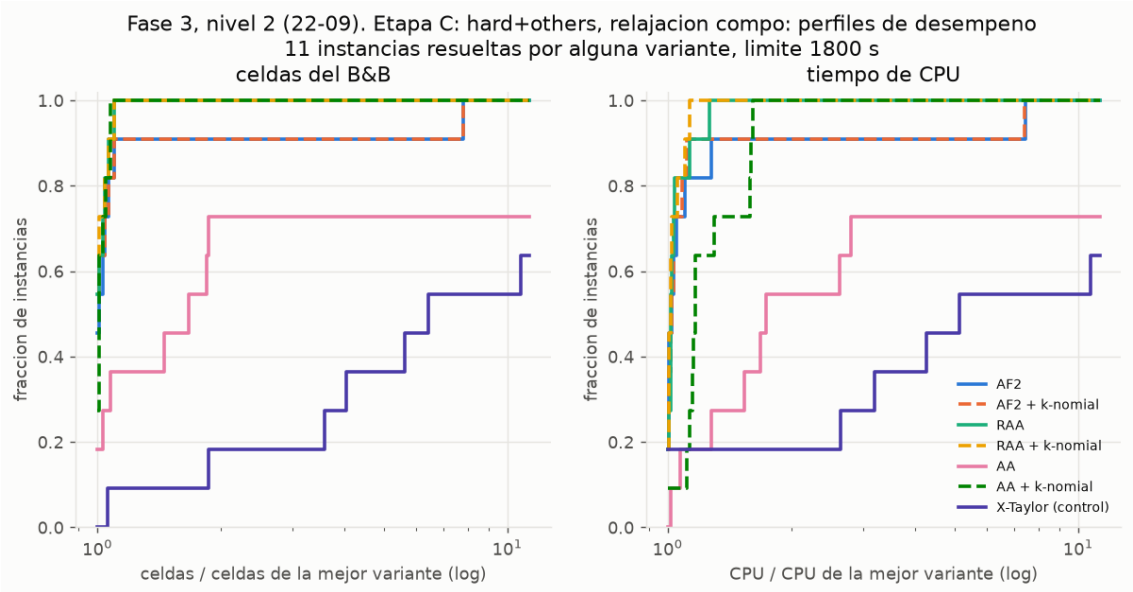


Figura 16. Etapa C: perfiles de desempeño.

Qué muestra. AF2, RAA y sus versiones con k-nomial resuelven las 11; **AA sola resuelve 8** (pierde `chem`, `chem-1` y `chembis`: agota los 1 800 s con 4 veces más celdas que AF2 en 420 s) y **AA + k-nomial vuelve a resolver las 11** con las mismas celdas que AF2 (111 720 frente a 111 578). xn resuelve 7 y es 2.8× peor en celdas. RAA: 0.830 en celdas y

CPU, explicado por una sola instancia, `ex8_4_5bis` (51 412 → 6 602 celdas, 46 → 6 s); en las otras 10 es idéntica a AF2 celda a celda.

Conclusión. En `hard` la relajación afín es decisiva, no un refinamiento. El k-nomial sobre AF2/RAA sigue sin actuar (0 / 11 / 0). Sobre AA deja de ser una mejora marginal: es la diferencia entre resolver o no.

Experimento 15 · Etapa D: hard + others, relajación `art` (15 × 6)

Datos	`results/07-fase3-nivel2-solver/fase3-20260921/D_hard_others_art/runs.csv`. Se detuvo a petición nuestra tras la C y se reanudó al día siguiente; las 12 corridas interrumpidas se rehicieron desde cero.

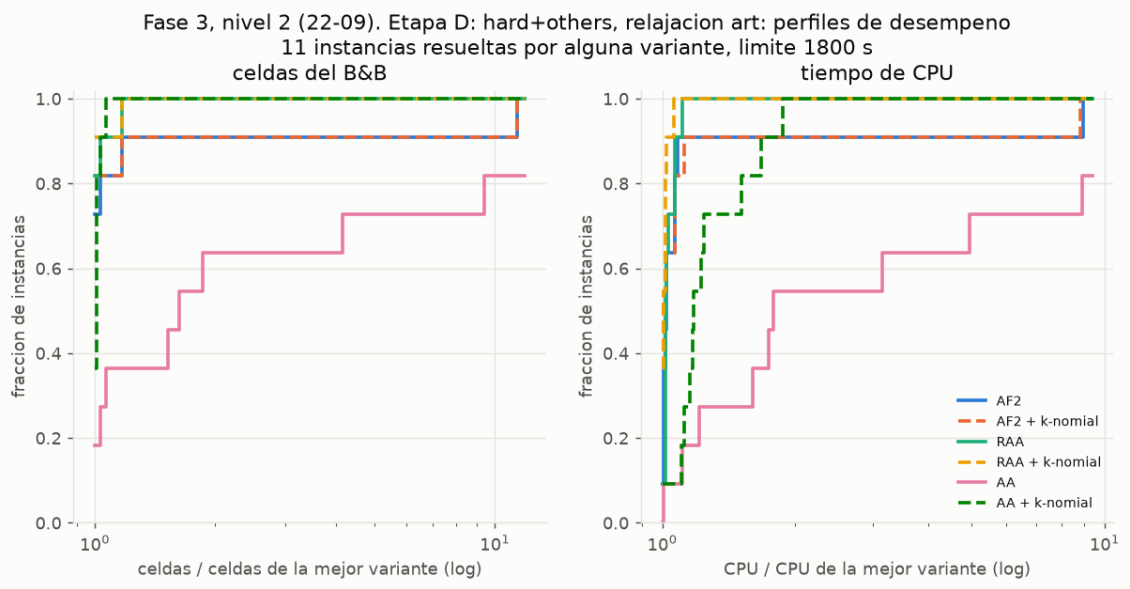


Figura 17. Etapa D: perfiles de desempeño.

Qué muestra. Mismo patrón que la C, más marcado: AA sola resuelve 9 (pierde `chem` y `chem-1` con 758 000 celdas frente a 112 836 de AF2) y es 35 % peor en celdas y 63 % en CPU en las que resuelve; con k-nomial resuelve las 11 y dentro de AA el efecto es 0.563 en celdas y 0.606 en CPU (5 gana / 4 empata / 0 pierde). RAA: 0.802 / 0.824 (`ex8_4_5bis`: 85 942 → 7 600). k-nomial sobre AF2/RAA: 0 / 11 / 0.

Conclusión. Con `art` sola no hay X-Taylor que amortigüe a la AA. Los resultados de C y D son solo 11 instancias comparables, así que hay que leerlos con esa cautela.

Resumen de las cuatro etapas

Qué es	Media geométrica de celdas y de CPU de cada variante frente a AF2 sin k-nomial, por etapa, sobre las instancias resueltas por ambas. `results/07-fase3-nivel2-solver/fase3-20260921/RESUMEN.md`.

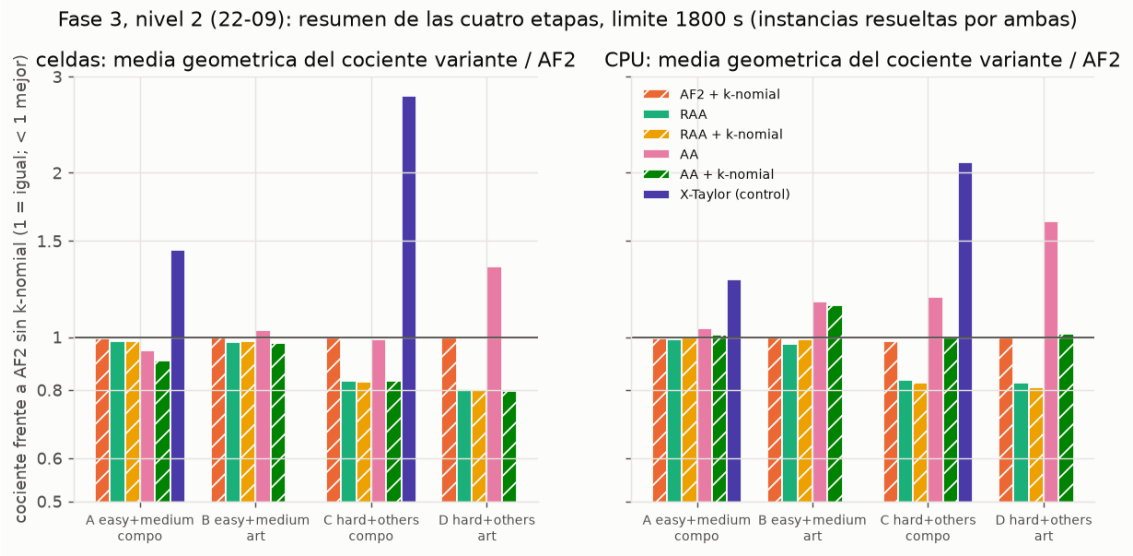


Figura 18. Las cuatro etapas en un gráfico. Rayado = con k-nomial.

Qué muestra. En celdas: AF2+k siempre pegado a 1; RAA y RAA+k iguales entre sí (el k-nomial no agrega nada a RAA), con 0.80–0.83 en hard; AA sola oscila entre 0.95 y 1.35; AA+k siempre bajo 1. En CPU: RAA es la única que baja el tiempo en las cuatro etapas; AA sola lo sube en todas (hasta 1.63); AA+k lo deja en 1.0 salvo en B.

Conclusión. Dos efectos limpios y uno nulo: RAA mejora o empata siempre y es gratis; el k-nomial sobre AA es consistente (0.56 a 0.96 en celdas); el k-nomial sobre AF2 y RAA es ruido (0.994 a 1.005).

5. Conclusiones

1. **Cotas (nivel 1).** RAA nunca es más ancha que AF2 y la mejora en el 3 % de las ternas, hasta 10 % en las funciones densas. AA gana y pierde a partes iguales frente a AF2 y es 15 veces más lenta por evaluación. El k-nomial casi no actúa sobre AF2 y RAA en `benchs/optim` y actúa 5 veces más sobre AA, sin ensanchar nunca. Cero violaciones de contención en 124 674 evaluaciones.
2. **Solver (nivel 2).** Las tres aritméticas afines dominan a X-Taylor (celdas ×0.69 en easy+medium, ×0.36 en hard). RAA es al menos tan buena como AF2 en las cuatro etapas y sin costo; sus tres victorias grandes (`ex8_4_4bis`, `bearing`, `ex8_4_5bis`, de 4 a 11 veces menos celdas) vienen de los cuadrados de Chebyshev, no del k-nomial.
3. **Qué aporta el k-nomial.** Sobre AF2 y RAA es neutro dentro del solver (0.994 a 1.005). Sobre AA es consistente: 0.957 y 0.948 en easy+medium, 0.78 y 0.56 en hard, y recupera tres instancias (`chem`, `chem-1`, `chembis`) que AA sola no resuelve en 30 min.
4. **Por qué es neutro sobre AF2/RAA (lo más importante que aprendimos).** El método está pensado para formas cuadráticas x^TQx completas. Dentro de IBEX se aplica en cada multiplicación de dos formas afines, y ahí la matriz $Q = (ab^T + ba^T)/2$ tiene siempre rango 2 y los factores casi nunca comparten dos símbolos. En `hard` hubo 5.4 millones de productos y ninguno extrajo un bloque. QPLIB lo confirma desde el otro lado: escrita monomio a monomio, 0 % de reducción en 58 de 58; con la forma completa, 2.8 % de media y hasta 10 %.
5. **Qué elegir hoy.** Con `compo` (la relajación por defecto): AA + k-nomial en celdas (0.905) con CPU igual a AF2, o RAA si importa el tiempo. Con `art` sola: RAA (0.979 celdas, 0.969 CPU). En easy+medium ninguna variante resuelve algo que otra no resuelva; las diferencias son de eficiencia, no de alcance.
6. **Seno y coseno.** La cuerda (Chebyshev de la RAA) domina al código original del plugin en todos los intervalos y conserva correlación; min-range da el rango exacto pero pierde pendiente. Las tres son válidas con extremos e inflexiones dentro gracias al mecanismo general de residuo.
7. **Limitaciones.** Una corrida por configuración (sin semillas alternativas); umbral ±5 % aproximado; margen de redondeo del k-nomial heurístico; solo 11 instancias comparables en hard; `blowup` y `unsolved` fuera del barrido por costo.

6. Qué funciona bien y qué hay que revisar

Funciona y es defendible:

- El port C++ del k-nomial es fiel al Python (bit a bit) y toda la aritmética nueva es válida (contención en todo lo probado). Donde no mejora, el resultado es idéntico al original.
- RAA en IBEX: nunca peor que AF2, gratis, con mejoras grandes donde hay cuadrados de formas compuestas.
- La cuerda para `sin`/`cos`, con el mecanismo general de residuo.
- La infraestructura: seis variantes conmutables, barridos reanudables, análisis automático y resultados reproducibles con su `meta.txt`.

Hay que revisar, por los resultados obtenidos:

- **Dónde se aplica el k-nomial.** En el producto binario la Q es de rango 2 y el método no tiene material. Para que actúe en el B&B hay que darle la forma cuadrática completa: un nodo cuadrático a nivel de expresión que acumule $x_0 + a^T\varepsilon + \varepsilon^TQ\varepsilon$ y la colapse una sola vez con el k-nomial (idea de Messine y Touhami, 2006), o reescribir los modelos cuadráticos antes de resolverlos. Esto se puede probar sin código nuevo: generar los `.bch` con los bloques ya extraídos y comparar `raa_k0` sobre el modelo original y el reescrito.
- **La cota del producto.** Para rango 2 existe la cota exacta de $(a^T\varepsilon)(b^T\varepsilon)$ sobre el cubo, en O(n log n) por el zonótopo proyectado en el plano (u, v). Dominaría a fAF2, a la forma simétrica y a cualquier k-nomial en ese nodo, y generaliza el cuadrado de Chebyshev que ya dio las victorias. Revisar novedad frente a Rump y Kashiwagi (2015).

- **Atribución en AA.** La ruta con k = 3 también introduce la simetrización y el término `Sxy/2` que la ruta original de `Affine3` no tiene. Falta la ablación AA con `max_k < 2` para saber cuánto del 0.56–0.96 es de los bloques y cuánto de la simetrización.
- **Unilateralidad.** El `AF_fAF2` del plugin tiene un solo error simétrico; el bloque `[0, s²]` se guarda como centro + radio y la siguiente operación pierde el signo. La AF2 de Messine tiene tres acumuladores ($e^{\pm}$, e^{+}, e^{-}). Implementarlos haría medible en C++ el par (E_inf, E_sup) del manuscrito.
- **Diseño experimental.** Repetir con 5 semillas (`--seed` ya existe en el CLI), fijar la cota superior inicial con el óptimo conocido para medir solo poder de acotación, y agregar un conjunto con estructura cuadrática real (BoxQP, QPLIB en Minibex).
- **Rigor.** El margen de redondeo de la reescritura es heurístico; calcular la discrepancia con la clase `Interval` lo cerraría con poco código (fase F6 pendiente).

7. Dónde está todo

Qué	Ruta
Repo y rama	`ibex-lib`, rama `raak-experimentos`, todo bajo `raak/`.
Plan, bitácora y documentos de cada fase	`raak/docs/PLAN_COMPARACION.md`, `FASE0_VERIFICACIONES.md`, `FASE2_TRIGONOMETRICAS.md`, `FASE3_EXPERIMENTOS.md`, `CAMBIOS_CODIGO.md` (cada función agregada), `ENTORNO.md` (cómo reconstruir el entorno).
Código	`raak/affine/ibex_KNomial.h`, `raak/patches/0001..0008`, `raak/cli/`, `raak/tests/`, `raak/scripts/`, `raak/analysis/`.
Resultados (CSV + análisis + gráficos)	`raak/results/`, una subcarpeta por tipo de experimento (índice en `raak/results/README.md`).
Gráficos de este informe	`raak/results/08-graficos/` (18 PNG + `README.md`).
Entorno de cómputo	WSL Ubuntu 24.04, `~/ibex-local/` (repos fijados, binarios, logs por corrida). Reconstrucción: `raak/scripts/wsl_build_all.sh`.

Glosario mínimo

Aritmética afín. Cada cantidad se representa como $x_0 + \Sigma\, a_i\varepsilon_i + \text{error}$, con $\varepsilon_i \in [-1,1]$ símbolos de ruido compartidos; conserva correlaciones que los intervalos pierden. **AF2** es la versión del plugin (vector denso + un error); **AA tradicional** crea un símbolo nuevo por cada operación no afín; **RAA** (revisada, paper [A]) acota mejor la parte cuadrática de productos y cuadrados. **k-nomial / GFW-k** (nuestro método): reescribe una forma cuadrática x^TQx como suma de cuadrados de sumas de hasta k variables más un residuo; cada cuadrado se acota en `[0, s²]`, mucho mejor que sus términos sueltos. **RAA-K** es ese método aplicado dentro de la multiplicación de la RAA.

B&B (branch and bound): parte la caja de las variables, calcula en cada sub-caja una cota inferior del objetivo con una relajación lineal (`xn` = X-Taylor; `art` = formas afines; `compo` = las dos intersectadas) y descarta la sub-caja si la cota es peor que la mejor solución conocida. **Celdas** = sub-cajas procesadas; menos celdas = mejores cotas. **Contención** = la forma afín contiene el rango real (lo obligatorio). **Sobreestimación** = cuánto más ancha es la forma que el rango real (lo deseable es que sea poca). **Media geométrica** de cocientes = promedio que no deja que una instancia enorme domine. **Perfil de desempeño** = qué fracción de instancias resuelve cada variante dentro de un factor X de la mejor.